Track Malfunctions in Flatland

TRailMix: Baruch Shteken[1] and Lena Friedek[2]

[1] Universität Potsdam, baruch.shteken@uni-potsdam.de
[2] Universität Potsdam, lena.friedek@uni-potsdam.de

Abstract. Malfunctions on and around tracks of a railway system are a common real-world situation. While train malfunctions render the agent immovable, issues on the track often allow a small window for decisions and rerouting of agents. We propose an implementation of track malfunctions for the Flatland framework leveraging environment solving with answer set programming. It investigates whether a secondary encoding for rescheduling of agents after a track malfunction is preferable to Redo Planning.

Keywords: ASP · Flatland · MAPF · Rescheduling · VRSP.

1 Introduction to Flatland and its malfunction handling

Flatland is a framework developed by AICrowd and SBB for multi-agent path finding problems and more specifically railway scheduling and rescheduling issues [7]. The initial simulation environment was designed for Python-based reinforcement learning approaches. It offers a contained experimentation sandbox around the vehicle rescheduling problem (VRSP) and simple two-dimensional visualisation of environments and agent movement.

1.1 Flatland and Answer Set Programming

Murphy has built a toolkit to integrate answer set programming solutions in the Flatland framework [9]. The approach solves instances with Clingo [4] and then translates the solution to execute and visualize in Flatland. Environments are generated beforehand to adhere to Flatland prerequisites but also encode the instance for Clingo. The ASP encoding for solving has to generate action predicates which communicate all agents' steps to Flatland.

1.2 Malfunctions in Flatland

Flatland environments can natively include agent malfunctions. The aspect is called Stochasticity and defines how often and for how long trains will be disrupted [8]. Murphy included this aspect in the script for building new environments [9]. The parameters in Code 1.1 define train malfunctions occurring in a new environment. The given `malfunction_rate` sets the Poisson rate so that we can expect 10 malfunctions over 30 time steps.

```
malfunction_rate=10/30
min_duration=2
max_duration=6
```

Code 1.1: Generate train malfunctions in build.py.

Murphy's interface between Clingo and Flatland allows to define separate encodings [9]. The first encoding derives the original schedule for all agents. The secondary encoding then solves the rescheduling after every train malfunction. It includes a feature for forwarding information from the scheduling encoding to be reused in the secondary encoding [9] [5].

Gilkra have used these features to benchmark different attempts for secondary encodings after train malfunctions [5]. They have been able reuse calculated plans from the scheduling step and thus reduce computing time during rescheduling.

Waldow approaches train malfunctions by identifying conflict points, cells where two agents can collide due to a train malfunction [10]. Deltas then define the distance of an agent to such a conflict point and help deduce whether a given train malfunction can be "waited out" (triviality) or if multiple agents have to wait. He calls replanning by solving the environment anew with the same base encoding "Redo Planning" [10, p. 24].

1.3 Track Malfunctions

Track malfunctions are a common real world issue in railway systems. It negatively affects punctuality and interferes with scheduling of trains. Deutsche Bahn reports old facilities in need of repairs just as well as an intense, high-frequent use of existing infrastructure as reasons for malfunctions around the track [2]. Other factors can be weather incidents, such as flooding [6] or fires [2], or trespassing individuals that make railway sections temporarily unusable [1].

2 Track Malfunctions in Flatland

Building on existing work about malfunctioning trains, we implement [3] track malfunctions to simulate realistic scheduling challenges (see Sections 1.2 and 1.3). We base our approach on previous work by Gilkra [5] and propose a similar structure for dealing with track-based disruptions in the original schedule. Flatland doesn't natively include track malfunctions. Alternatively, we generate track malfunctions in the main solving pipeline (see Section 2.2). By reusing plans for trains from the original schedule, we aimed to improve computation time during rescheduling through a secondary encoding (see Section 2.3).

2.1 Base Encoding

The base encoding is Clingo code that solves a Flatland environment and schedules agents. It sets up the environment, generates sequential and valid positions

for agents and then translates the positions into action atoms for Flatland (see Figure 1).

Setting-up the environment The environment specifications are defined for Clingo during environment creation with `build.py`. It consists of a global time limit, information about the trains including agent ID, start, end and speed, and the cells with their given track type.

Our Base Encoding includes rules and additional information to represent the environment. First, it imports a fact file (`connections_nswe.lp`) with additional world information applicable to all Flatland environments. It defines how track types translate to the directions north, south, west and east. For instance, the fact `connection(4608, (s,w))` is track type 4608, a curve that connects south and west. Furthermore, it contains definitions for directions and turns, a move in a certain direction in terms of movement on the X- and Y-axis, relations between directions (e.g. west is the inverse of east) and which track types allow a choice in direction. The fact file has been taken from Murphy [9] in big parts and expanded by us. Then, the Base Encoding creates a grid of adjacent cells and checks which are reachable by an agent.

Move Agents The `position(ID, CELL, TIME, DIRECTION)` argument handles the position of a train (ID) in the environment at any given time step and thus generates the sequential moves for that agent. A train is marked as `finish(ID, TIME)` once it has reached its destination and doesn't move further. The position atoms are subsequently translated to action predicates containing necessary information, namely train ID and the required action at a given time step, for Flatland to visualise the solved environment.

Our optimization objectives encourage trains to reach their destination as soon as possible and prefer one consecutive waiting period over repeated stop-and-go behaviour.

Constraints We implemented two types of constraints on agent movements: single train constraints and train coordination constraints. Single train constraints ensure that a train behaves normally. They forbid multiple positions of one agent in the environment and ensure termination upon arrival at the destination.

Train Coordination constraints avoid behaviour that would cause collisions between agents. Any two given trains cannot occupy the same cell and two oncoming agents cannot immediately switch places or pass through each other.

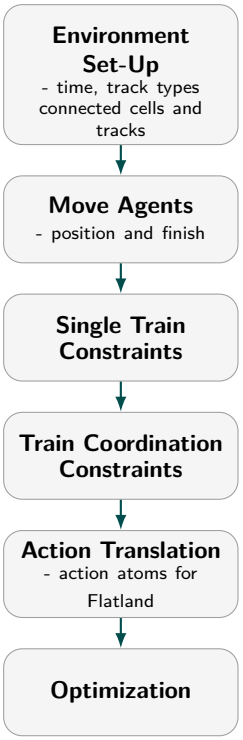


Fig. 1: Overview of Base Encoding.

2.2 Python Part

Since we could not implement track malfunctions directly in Flatland, we implemented them in Python code that runs Clingo with Flatland input (`solve.py`).

The added code consists of the following parts:

- Creating track malfunctions either randomly or deterministically
- Translating the track malfunction Python object into Clingo
- Forwarding the atoms to Clingo
- Logging the malfunction in a file
- Depicting the malfunction in the Flatland output

Track Malfunction Atom Definition A track malfunction Python object is a tuple consisting of three components:

```
(CELL, TIMESTEP, DURATION)
```

The `CELL` is an `(X, Y)` tuple that defines where the track malfunction occurred. The `TIMESTEP` represents the timestep at which the track malfunction was triggered. During this timestep, the malfunction has not yet occurred but it starts in the following timestep. This was an architectural decision to ensure that trains cannot be on a malfunctioning track when the malfunction starts. The `DURATION` specifies the number of timesteps for which the track malfunction occurred.

Track malfunctions can either be created randomly or predefined by the user. The variable `predefined_malfunction` is used to store malfunctions that the user wants to enforce on the environment. If the variable is empty, there is a 1% chance of a malfunction occurring at every timestep (this probability can be adjusted). In the current setup, only one malfunction can occur per timestep (see Section 6). Malfunctions are logged in a CSV file with the following parameters:

```
timestep;cell;duration
```

Track malfunction Implementation in Clingo When a malfunction is triggered in Python, multiple atoms and rules are generated and forwarded to the program that calculates the rescheduling (Secondary Encoding - see Section 2.3).

- **`action` constraint rules** – To preserve the actions taken up to this point, a constraint rule is generated for each action, ensuring that all past actions must be present in the output of the rescheduling.
- **`track_malfunction` atoms** – to provide the location, timestep and duration of the malfunction.
- **`position` constraint rules** – Since the track malfunction starts one timestep after it is triggered, the actions at the time the malfunction is triggered are preserved. Consequently, their positions in the next timestep are also preserved. These constraints only include the next position.

– **`planned_action` atoms** – Include all actions that were planned before the malfunction occurred.
– **`planned_position` atoms** – Include all positions that were planned before the malfunction occurred.
– **`current_position` atoms** – Include all positions occupied at the timestep at which the malfunction occurred.

Track malfunction Depiction Similar to a train malfunction, a track malfunction is depicted as a red **X** over the malfunctioning track.

2.3 Secondary Encoding

The secondary encoding reschedules the environment after a track malfunction. Our goal was to reduce computational cost by avoiding recalculation of the environment and agents. Our approach, similar to Gilkra [5], reuses information and train schedules from the original plan unless the track malfunction interferes with an agent.

`secondary_encoding.lp` reuses ideas for solving from the base encoding, such as the general environment facts `connections_nswe.lp` and `trainjacent` atoms for finding valid movements (see Section 2.1). Additionally, information is passed from the original schedule (see Section 2.2). Thus, the secondary encoding has access to

– basic Flatland information (`connections_nswe.lp`)
– details of the track malfunction (`track_malfunction\3`)
– agent history (`action` and `position` constraints, `current_position\4`)
– initial plan for agents (`planned_action\3` and `planned_position\4`)

First, the program sets up the required environment with `connections_nswe.lp` and creates two time counters. `time(T)` counts time from one step after the track malfunction. `past_time(T)` counts time up until and including the step of the track malfunction. This distinction is required to restore actions up until the malfunction time. These past action atoms are necessary for Flatland to depict the solution correctly.

If a track malfunction triggers before all trains of the environment have been initialized in their starting position, they have to be positioned during rescheduling by creating their starting `position\4`. Then the replanning of trains starts. To identify whether an agent is affected by a track malfunction, it raises a `consequence\3` (see Code 1.2). That means the train is planned to be on the malfunctioning track, it cannot

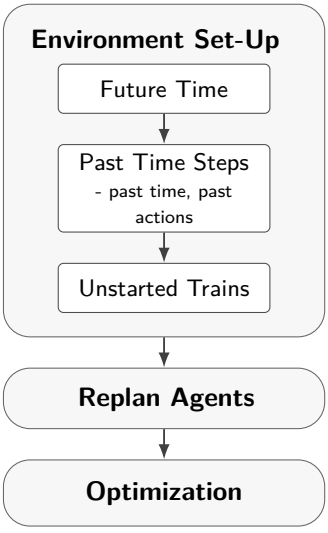


Fig. 2: High-level overview of Secondary Encoding.

continue on its original path and must adjust its behaviour. There are three scenarios our secondary encoding handles[3]: a) the track malfunction doesn't interfere with the planned routes of agents (no consequence), b) one train is affected by the track malfunction (simple consequence) and c) multiple trains are affected by the track malfunction (complex consequence).

```
1
2  % Consequence: train is affected by a track malfunction and
     has to be recalculated
3  consequence(train(ID), P, T) :- planned_position(ID, P, T,
     _), track_malfunction(P, Dur_mal, T_mal), Dur_mal + T_mal
     >= T.
4
5  % planned is a train that is not affected by the
     malfunction and we don't want to reroute it
6  {planned(ID)}:- train(ID), not consequence(train(ID), _, _).
7
```

Code 1.2: Definition of consequence: if a train is attempting to go into a malfunctioning cell it raises a consequence.

No Consequence If no agent is planned to visit the actively malfunctioning cell the trains can continue their paths as calculated in the base encoding. All agents are marked as `planned\1`. Their planned actions and positions are translated into `action\3` and `position\4` atoms.

Simple Consequence In a simple consequence, only one train is scheduled to pass the track malfunction (only one `consequence/3` atom). The train gets marked as `calculate/1`. Other trains remain `planned/1`. From its current position this agents gets rescheduled. The rescheduling is done similarly to the scheduling logic in the base encoding by checking reachable cells, creating `position` and `action` atoms, narrowing down the solution with constraints and selecting the best model via optimization statements (see Section 2.1). A rescheduled train can "wait out" the malfunction or be rerouted via other available tracks.

Complex Consequence A complex consequence includes multiple agents that require rescheduling (multiple `conequence/3` atoms). However, it also covers the case where the adjustment of one agent's path influences another train, e.g. the rerouted or waiting train would cause a collision with an unaffected train. The trains that were planned as passing through the track malfunction are marked with `consequence/3`. Those agents and the ones indirectly affected are marked as `calculate/1`. Other trains remain `planned/1`. The rescheduling then is done as described above.

[3] If a track malfunction hits a cell while there is a train on its track, the simulation breaks, effectively corresponding to an accident on the track. We discovered this behaviour and decided to keep it instead of handling that exception as it also depicts a realistic scenario and could be inspiration for future work in this course.

3 Experiments

We have conducted four experiments to test the capabilities of our encodings in multiple environments. Clingo reports total elapsed time and time spent on solving. For each experiment, we assume that most of the non-solving time is used to ground the rules. We base our comparisons on time differences in computing since rescheduling is a time-sensitive interruption with only a limited window for decision making.

Experiment 1: Effect of the Heuristic Subsequently, a heuristic was added to the secondary encoding (Line 6 in Code 1.3). Due to the size of most Flatland environments and the low rate of track malfunctions (1%), we could assume that most malfunctions cause no or simple consequences. Clingo should be able to transfer the original plan for planned trains rather quickly. However, we had to encourage and direct this behaviour in Clingo with a heuristic. Experiment 1 compares performance of the secondary encoding with and without the heuristic prioritizing planned trains.

```
1
2  % planned is a train that is not affected by the
     malfunction and we don't want to reroute it
3  {planned(ID)}:- train(ID), not consequence(train(ID), _, _).
4
5  % Prioritize planned trains when solving
6  #heuristic planned(ID): train(ID). [100, true]
7
```

Code 1.3: The heuristic flag in line 6 prioritizes solutions with planned trains.

Experiment 2: Base Encoding vs. Secondary Encoding Experiment 2 investigates if rescheduling with the secondary encoding is indeed preferable over replanning with the Base Encoding (Redo Planning).

Experiment 3: Base Encoding vs. Secondary Encoding with Time Prioritization The Secondary Encoding contains three optimization statements (see Code 1.4). In Experiment 3, we seek to test if prioritizing time can still lead to a more efficient rescheduling than Redo Planning. The Base Encoding also prioritizes time via its optimization statements (see Section 2.1). The Secondary Encoding that prioritizes time differs from the Secondary Encoding + heuristic in such that it switches the weights for the two top-most optimization statements. The heuristic flag is included in both versions. In the time prioritization the weights are as shown in Code 1.4.

```
1
2  % we want as many OG train plans as possible
3  #maximize { 1@2, ID : planned(ID)}.
```

```
4
5  % Finish as early as possible
6  #minimize { 1@3, ID, T : action(train(ID), _, T) }.
7
8  % Prefer fewer distinct waiting episodes
9  wait_start(ID, T) :- action(train(ID), wait, T),
     action(train(ID), A, T-1), A != wait.
10
11 #minimize { 1@1, ID, T : wait_start(ID, T) }.
12
```

Code 1.4: Optimization statements in Secondary Encoding + time prioritization.

Experiment 4: Heuristic vs. Time Prioritization Experiment 4 then evaluates how the Secondary Encoding + heuristic competes against Secondary Encoding + time prioritization. Thus, we can determine the best version of the Secondary Encoding for rescheduling after a track malfunction.

3.1 Testing environments

Six testing environments of varying sizes and numbers of agents were used in each experiment (see Figure 3). In each environment, we have identified tracks for the three types of track malfunctions (simple, complex, no consequence - see Section 2.3).

3.2 Predictions

The output for each encoding when rescheduling after a malfunction will not necessarily be the same. The base encoding and the secondary encoding with time prioritization will output the optimal solution, meaning that the trains will spend the least possible amount of time on the network overall.

However, the secondary encoding with and without heuristics prioritizes the preservation of the planned schedule. This means that even if delaying a non-consequent train would improve the overall travel time, only the base encoding and the secondary encoding with time prioritization will derive this solution. The other two encodings will not be able to choose this solution.

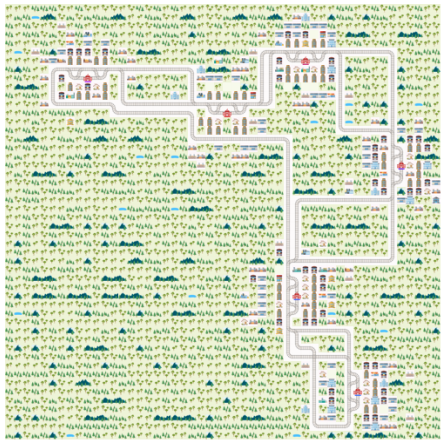
4 Results[4]

4.1 Experiment 1: Heuristic vs. no Heuristic

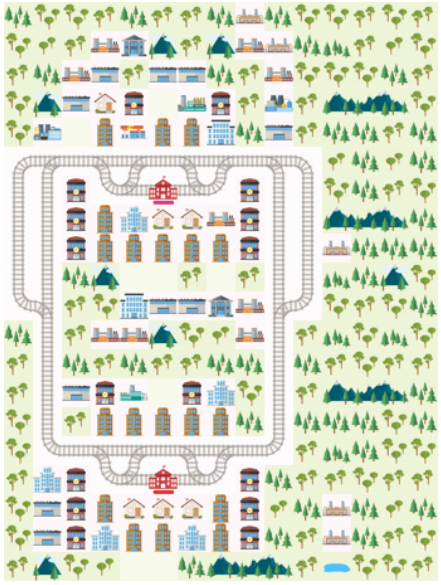
Results are reported in Table 1. A positive ΔT_{res} indicates that the encoding with the heuristic is faster. The encoding with the heuristic is significantly faster in all environments except `env_3_2` and `env_2_2`. In some runs of those instances,

[4] The raw results are listed in Appendix A.

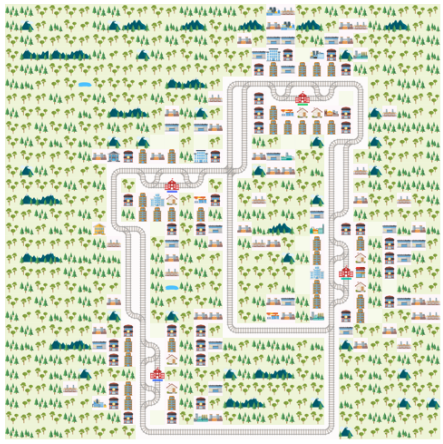
(a) env_3_2, 20x20, 3 agents



(b) env_6_6, 50x50, 6 agents



(c) env_2_2, 20x15, 2 agents



(d) env_10_10, 30x30, 10 agents

(e) env_5_4, 25x40, 5 agents

(f) env_10_8, 25x40, 10 agents

Fig. 3: Testing environments.

the encoding without the heuristic performed marginally better. This difference is mostly negligibly small.

The biggest differences can be found in `env_6_6` and `env_10_8` - the biggest and densest environments tested. Especially, for the no consequence scenario the heuristic can significantly enhance calculation time. In those instances, any secondary encoding needs to re-prove that the original schedule was optimal. Without the heuristic, Clingo starts with an initial solution that recalculated all the trains and, after many iterations, eventually reached the original planned solution.

With the heuristic flag, Clingo assumes that most trains should use the original planned solution. We assume that since the secondary encoding has more rules and options, it is harder to solve without redirecting Clingo toward the planned train solution.

Environment	**Malfunction type**	**heuristic**	**no heuristic**	ΔT_{res}
env_2_2	Non-consequent	0.071	0.042	-0.029
	Consequent 1	0.032	0.032	0.000
	Consequent + collisions	0.048	0.047	-0.001
env_3_2	Non-consequent	0.116	0.097	-0.019
	Consequent 1	0.071	0.091	+0.020
	Consequent 2	0.084	0.054	-0.030
	Consequent + collisions	0.070	0.075	+0.005
env_5_4	Non-consequent	0.576	5.142	+4.566
	Consequent 1	0.561	7.897	+7.336
	Consequent 2	0.389	0.421	+0.032
	Consequent + collisions	0.867	2.722	+1.855
env_6_6	Non-consequent	10.794	256.370	+245.576
	Consequent 1	9.941	206.032	+196.091
	Consequent 2	10.245	90.477	+80.232
	Consequent + collisions	25.852	57.022	+31.170
env_10_8	Non-consequent	4.508	110.196	+105.688
	Consequent 1	1.231	17.976	+16.745
	Consequent 2	1.344	31.010	+29.666
env_10_10	Non-consequent	1.303	83.612	+82.309
	Consequent 1	1.610	96.502	+94.892
	Consequent 2	0.509	0.888	+0.379
	Consequent + collisions	2.587	71.364	+68.777

Table 1: Comparison of total rescheduling times, T_{res}, for secondary + heuristic and secondary without heuristic. The difference is computed as $\Delta T_{\text{res}} = T_{\text{res}}^{\text{secondary, no heuristic}} - T_{\text{res}}^{\text{secondary+heuristic}}$. Positive values indicate that using the heuristic is faster than without the heuristic.

4.2 Experiment 2: Base Encoding vs. Secondary Encoding with Heuristics

Results are reported in Table 2. Negative ΔT_{res} indicate that the secondary encoding is faster. For smaller environments, like `env_2_2` and `env_3_2`, the running time is low and very similar for both encodings. For larger environments, the secondary encoding with heuristics is significantly faster. For example, in `env_6_6`, the differences in the running time were substantial. In the non-consequent malfunction, the secondary encoding with heuristics ran two minutes faster than the base encoding. The secondary encoding makes use of the forwarded information and can find an optimal solution more efficiently than the base encoding (Redo Planning).

Environment	**Malfunction type**	**Base** T_{res}	**Secondary** T_{res}	ΔT_{res}
env_2_2	Non-consequent	0.054	0.071	+0.017
	Consequent 1	0.040	0.032	-0.008
	Consequent + collisions	0.053	0.048	-0.005
env_3_2	Non-consequent	0.107	0.116	+0.009
	Consequent 1	0.104	0.071	-0.033
	Consequent 2	0.082	0.084	+0.002
	Consequent + collisions	0.089	0.070	-0.019
env_5_4	Non-consequent	0.655	0.576	-0.079
	Consequent 1	0.635	0.561	-0.074
	Consequent 2	0.607	0.389	-0.218
	Consequent + collisions	3.747	0.867	-2.880
env_6_6	Non-consequent	138.697	10.794	-127.903
	Consequent 1	127.527	9.941	-117.586
	Consequent 2	97.934	10.245	-87.689
	Consequent + collisions	100.216	25.852	-74.364
env_10_8	Non-consequent	24.704	4.508	-20.196
	Consequent 1	7.952	1.231	-6.721
	Consequent 2	25.222	1.344	-23.878
env_10_10	Non-consequent	44.581	1.303	-43.278
	Consequent 1	97.233	1.610	-95.623
	Consequent 2	1.872	0.509	-1.363
	Consequent + collisions	71.720	2.587	-69.133

Table 2: Comparison of total rescheduling times for base and secondary + heuristic. Negative values of ΔT_{res} indicate that secondary + heuristic is faster than base.

4.3 Experiment 3: Base Encoding vs. Secondary Encoding with Time Prioritization

Table 3 reports the results for experiment 3. Negative ΔT_{res} values indicate that the secondary encoding with time prioritization is faster than the base encoding (Redo Planning). The base encoding is faster or equally as fast (marginal difference) as the compared encoding when there is no consequence. The secondary encoding with time prioritization is significantly faster when there is a consequence. Especially in scenarios with complex consequences the secondary encoding's calculation time is much lower, e.g. in `env_10_10`, a medium environment with dense agent activity, the secondary Encoding with time prioritization can reduce computation time by more than 50%.

Environment	**Malfunction type**	**base**	**time prioritization**	ΔT_{res}
env_2_2	Non-consequent	0.054	0.050	-0.004
	Consequent 1	0.040	0.031	-0.009
	Consequent + collisions	0.053	0.045	-0.008
env_3_2	Non-consequent	0.107	0.099	-0.008
	Consequent 1	0.104	0.093	-0.011
	Consequent 2	0.082	0.058	-0.024
	Consequent + collisions	0.089	0.076	-0.013
env_5_4	Non-consequent	0.655	0.983	+0.328
	Consequent 1	0.635	0.807	+0.172
	Consequent 2	0.607	0.585	-0.022
	Consequent + collisions	3.747	1.132	-2.615
env_6_6	Non-consequent	138.697	179.266	+40.569
	Consequent 1	127.527	87.676	-39.851
	Consequent 2	97.934	67.346	-30.588
	Consequent + collisions	100.216	71.626	-28.590
env_10_8	Non-consequent	24.704	39.227	+14.523
	Consequent 1	7.952	1.735	-6.217
	Consequent 2	25.222	5.410	-19.812
env_10_10	Non-consequent	44.581	9.817	-34.764
	Consequent 1	97.233	66.304	-30.929
	Consequent 2	1.872	0.740	-1.132
	Consequent + collisions	71.720	24.082	-47.638

Table 3: Comparison of total rescheduling times, T_{res}, for base and secondary + time prioritization. The difference is computed as $\Delta T_{\text{res}} = T_{\text{res}}^{\text{secondary+time priority}} - T_{\text{res}}^{\text{base}}$. positive values indicate that base is faster.

4.4 Experiment 4: Time vs. first plan prioritization

Table 4 reports the results from experiment 4. Positive ΔT_{res} values indicate that the encoding prioritizing the first plan (or least number of train changes) is faster. The secondary encoding that prioritizes the first plan is faster in almost all environments and scenarios. The differences between the encodings are negligible in small environments, like `env_3_2` and `env_2_2`. However, in larger environments the secondary encoding prioritizing first plans is significantly more efficient. The difference is particularly pronounced in the largest environment `env_6_6` with the first plan encoding being almost 3 minutes faster.

Environment	Malfunction type	heuristic	time priority	ΔT_{res}
env_2_2	Non-consequent	0.071	0.050	-0.021
	Consequent 1	0.032	0.031	-0.001
	Consequent + collisions	0.048	0.045	-0.003
env_3_2	Non-consequent	0.116	0.099	-0.017
	Consequent 1	0.071	0.093	+0.022
	Consequent 2	0.084	0.058	-0.026
	Consequent + collisions	0.070	0.076	+0.006
env_5_4	Non-consequent	0.576	0.983	+0.407
	Consequent 1	0.561	0.807	+0.246
	Consequent 2	0.389	0.585	+0.196
	Consequent + collisions	0.867	1.132	+0.265
env_6_6	Non-consequent	10.794	179.266	+168.472
	Consequent 1	9.941	87.676	+77.735
	Consequent 2	10.245	67.346	+57.101
	Consequent + collisions	25.852	71.626	+45.774
env_10_8	Non-consequent	4.508	39.227	+34.719
	Consequent 1	1.231	1.735	+0.504
	Consequent 2	1.344	5.410	+4.066
env_10_10	Non-consequent	1.303	9.817	+8.514
	Consequent 1	1.610	66.304	+64.694
	Consequent 2	0.509	0.740	+0.231
	Consequent + collisions	2.587	24.082	+21.495

Table 4: Comparison of total rescheduling times, T_{res}, for Secondary + heuristic and Secondary + time priority. The difference is computed as $\Delta T_{\text{res}} = T_{\text{res}}^{\text{secondary+time priority}} - T_{\text{res}}^{\text{secondary+heuristic}}$. Positive values indicate that Secondary + heuristic is faster.

5 Discussion

Our proposed secondary encoding including a heuristic for prioritizing the least amount of changes improves running time compared to Redo Planning with

the base encoding. The heuristic improves total calculation time significantly. However, by prioritizing the least amount of changes qualitative details can be missed.

Figure 4 illustrates this discrepancy. It shows snapshots from two rescheduling solutions: the secondary encoding with heuristic (left) and the secondary encoding prioritizing time (right). Part of this environment is a purple train (as shown in Figure 4a). A malfunction is triggered from time step 70 to 90 (see Figure 4b and 4c). The heuristic prioritizes changing the least amount of trains which is why the purple train waits for 29 time steps on track (6,6) (see Figure 4b). The westmost station is connected to the station northeast from it by only one track. Thus, the purple agent has to wait for the green train (see Figure 4d) because it enters this section earlier. In the alternative encoding, both the red and the green agent disrupt their journey slightly by one and six seconds, respectively. The purple continues its journey without delay and can pass the green train ten seconds earlier (see Figure 4e).

In the given example, time prioritization offers the better solution in terms of total waiting time of agents (7s vs. 29s) and total time steps in the environment (136 vs. 146). It disrupts the plans of more trains but for shorter periods. In real-life application time prioritization would be the qualitatively preferable solution. It facilitates higher customer satisfaction, shortest total delay time across agents and therefore better chances for further connections.

However, the computational costs of both encodings differ significantly. The heuristic approach was much faster (4.508s) than the prioritization of time (39.227s, see Table 3). Time prioritization thus requires a longer reaction time after the registration of a track malfunction.

5.1 Encoding Selection After a Track Malfunction

There are two possible goals that a user might want when rescheduling after a malfunction:

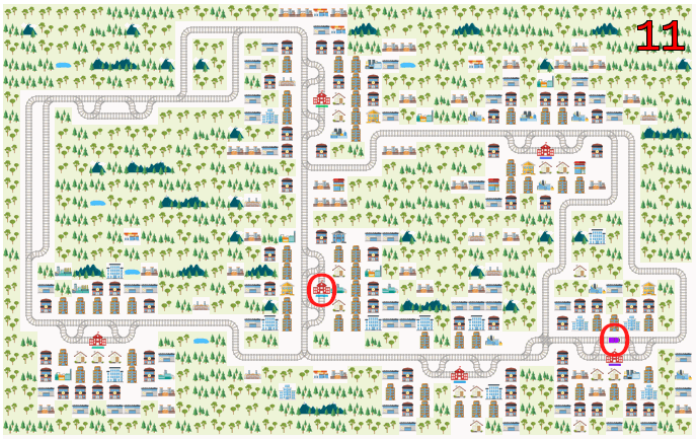
- Change the least amount of trains possible, i.e., preserve the previous schedule as much as possible.
- Obtain the most optimized solution.

Both goals should be achieved as quickly as possible.

For the purpose of comparing the encodings, we assume that the output of the initial schedule using the base encoding is already saved. This assumption reflects the real-world scenario in which the schedule is set in advance and the malfunction occurs later.

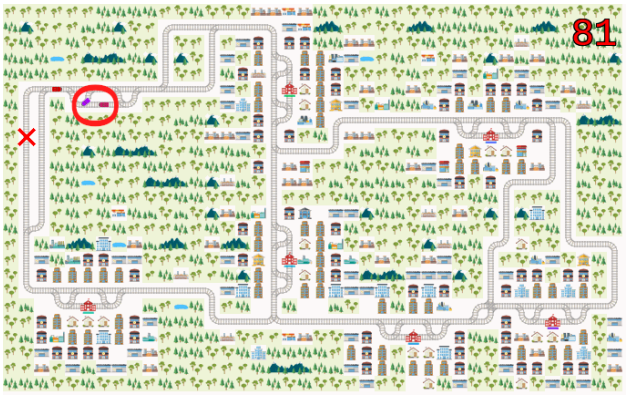
For obtaining the most optimized solution, the base encoding and the secondary encoding with time prioritization are the preferred encodings. Conversely, the secondary encoding with and without heuristics are used to produce an answer with the least amount of changed trains.

For smaller environments, defined as a height below 20, a width below 20, and fewer than 3 trains, all encodings are very fast, and no malfunction requires more than 5 seconds to find a solution.

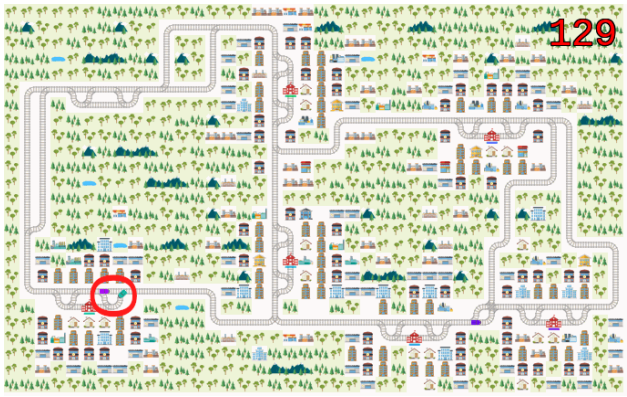


(a) The purple train starts in (19,35) for both solutions and terminates at the station marked in red.

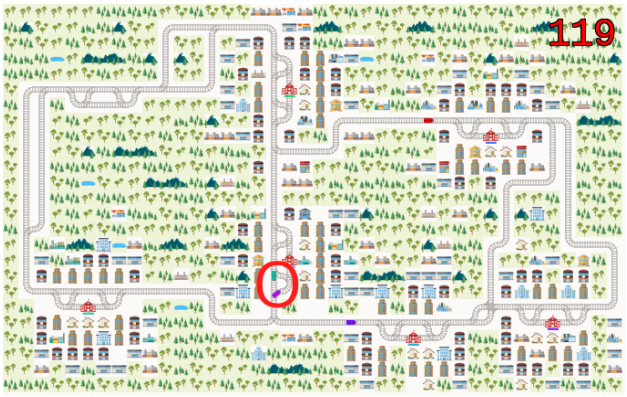
(b) Heuristic. The purple train waits in (6,6) for 29 time steps.



(c) Time Prioritization. The purple train continues. The red train waits for one time step.



(d) Heuristic. The purple and green train pass each other at time step 129.



(e) Time Prioritization. The purple and green train pass each other at time step 119.

Fig. 4: Qualitative comparison of rescheduling with heuristic/least changes (left) and time prioritization (right).

For larger environments, however, the differences between the encodings become significant. If the user wants to obtain the least amount of changed trains possible, they should choose the secondary encoding with heuristics, as it is consistently the fastest encoding, often by a large margin.

If the user wants the most optimized solution, the preferred encoding depends on whether there is a consequence. If there is a consequence, the secondary encoding with time prioritization is the better choice in most cases. Otherwise, the base encoding is faster.

Combining all the conclusions derived above, we can create a flow that describes how to handle a malfunction. First, the secondary encoding with heuristic is run to derive aolution A. If there is no consequence, solution A is both the most optimized solution and the solution with the least amount of changed trains. If there is a consequence, the appropriate solution depends on the user's goal. If the goal is to minimize the number of changed trains, solution A can be used directly. If the goal is to obtain the most optimized solution, the secondary encoding with time prioritization is run to derive solution B. Most malfunctions will not have a consequence, so it is better to run the secondary encoding with heuristic first and determine whether rescheduling is necessary. The secondary encoding with heuristic is also very fast in all cases, so an answer will be available before the other encodings.

Using the secondary encoding with time prioritization only when a consequence is detected was faster than the base encoding in all tested cases.

A flowchart depicting which encoding should be used based on the desired output can be viewed in Figure 5.

6 Future Work

Future work could consider other approaches than uniform randomness as a decider for a track malfunction. For instance, malfunctions on highly frequented tracks or crossings could be more likely than on straight connections.

In our current pipeline, scheduling and rescheduling are tied together tightly. For benchmarking of different encoding options, it can be useful to separate out these two steps. Such a set-up would have to save the scheduling plan from the base encoding, then create a malfunction and call the plan again for generating the information to forward for rescheduling. This approach could ease comparisons between rescheduling encodings by using the exact same model from the base encoding.

Additionally, our approach currently only handles one emerging track malfunction at a given time step and overlapping malfunctions with different starting times. Future projects could consider adjusting `solve.py` to handle two or more simultaneous track malfunctions.

Finally, we noticed that sometimes a malfunction can cause an environment to be unsatisfiable, i.e. when a malfunction hits a particularly crucial track. Then the program crashes without any visual output despite having successfully calculated the original schedule. Future projects could focus on handling this

issue and generating the base plan in Flatland or maybe even visualizing when and why the program fails.

References

1. Bahn-Verspätungen durch Menschen an und auf Gleisen. Süddeutsche Zeitung (2023), https://www.sueddeutsche.de/wirtschaft/statistik-bahn-verspaetungen-durch-menschen-an-und-auf-gleisen-dpa.urn-newsml-dpa-com-20090101-230610-99-05514, Accessed: 18.08.2026
2. Integrierter Zwischenbericht 2025. Deutsche Bahn AG (2025), https://zbir.deutschebahn.com/2025/de/konzern-zwischenlagebericht-ungeprueft/qualitaet-und-sicherheit/puenktlichkeit/, Accessed: 10.08.2026
3. Friedek, L., Shteken, B.: Trailmix. https://github.com/wug-on-praat/TRailMix (2026), gitHub repository
4. Gebser, M., Kaminski, R., Kaufmann, B., Schaub, T.: Multi-shot asp solving with clingo (2018), https://arxiv.org/abs/1705.09811
5. Gili i Bueno, G., Kratzsch, F.: Flatland: Malfunctions research on train training (2025), https://github.com/krr-up/railway-research/blob/main/railway-course/2024-wise/gilkra2025a.pdf
6. Loderbauer, G.: Punctuality in rail transport in Autria in 2024. Agentur für Passagier- und Fahrgastrechte (2025), https://en.apf.gv.at/news-detail/P%C3%BCnktlichkeit2024, Accessed: 18.08.2026
7. Mohanty, S., Nygren, E., Laurent, F., Schneider, M., Scheller, C., Bhattacharya, N., Watson, J., Egli, A., Eichenberger, C., Baumberger, C., Vienken, G., Sturm, I., Sartoretti, G., Spigler, G.: Flatland-rl: Multi-agent reinforcement learning on trains (2020), https://arxiv.org/abs/2012.05893
8. Mohanty, S., Nygren, E., Laurent, F., Schneider, M., Scheller, C., Bhattacharya, N., Watson, J., Egli, A., Eichenberger, C., Baumberger, C., Vienken, G., Sturm, I., Sartoretti, G., Spigler, G.: Flatland book (2023), https://flatland-association.github.io/flatland-book/environment/environment/stochasticity.html, Accessed: 19.08.2026
9. Murphy, K.R.: Flatland (2026), https://github.com/krr-up/flatland, Accessed: 17.08.2026
10. Waldow, I.: Intelligent malfunction handling in large railway systems (2025), https://github.com/krr-up/railway-research/blob/main/theses/waldow2025a.pdf

A Experiment Results

Environment	Malfunction type	Rescheduling type	T_{ground}	T_{solve}	T_{res}
env_2_2					
	Non-consequent	Base	0.034	0.02	0.054
		Secondary + heuristic	0.071	0	0.071
		Secondary + time priority	0.040	0.01	0.050

Continued on next page

Table 5 – continued from previous page

Environment	Malfunction type	Rescheduling type	T_{ground}	T_{solve}	T_{res}
		Secondary, no heuristic	0.042	0	0.042
	Consequent 1	Base	0.040	0	0.040
		Secondary + heuristic	0.032	0	0.032
		Secondary + time priority	0.031	0	0.031
		Secondary, no heuristic	0.032	0	0.032
	Consequent + collisions	Base	0.043	0.01	0.053
		Secondary + heuristic	0.038	0.01	0.048
		Secondary + time priority	0.035	0.01	0.045
		Secondary, no heuristic	0.037	0.01	0.047
env_3_2					
	Non-consequent	Base	0.077	0.03	0.107
		Secondary + heuristic	0.116	0	0.116
		Secondary + time priority	0.079	0.02	0.099
		Secondary, no heuristic	0.077	0.02	0.097
	Consequent 1	Base	0.074	0.03	0.104
		Secondary + heuristic	0.071	0	0.071
		Secondary + time priority	0.083	0.01	0.093
		Secondary, no heuristic	0.081	0.01	0.091
	Consequent 2	Base	0.072	0.01	0.082
		Secondary + heuristic	0.084	0	0.084
		Secondary + time priority	0.058	0	0.058
		Secondary, no heuristic	0.054	0	0.054
	Consequent + collisions	Base	0.079	0.01	0.089
		Secondary + heuristic	0.060	0.01	0.070
		Secondary + time priority	0.056	0.02	0.076
		Secondary, no heuristic	0.065	0.01	0.075
env_5_4					
	Non-consequent	Base	0.545	0.11	0.655
		Secondary + heuristic	0.566	0.01	0.576
		Secondary + time priority	0.653	0.33	0.983
		Secondary, no heuristic	0.692	4.45	5.142
	Consequent 1	Base	0.455	0.18	0.635
		Secondary + heuristic	0.541	0.02	0.561
		Secondary + time priority	0.597	0.21	0.807
		Secondary, no heuristic	0.587	7.31	7.897
	Consequent 2	Base	0.507	0.10	0.607
		Secondary + heuristic	0.359	0.03	0.389
		Secondary + time priority	0.395	0.19	0.585
		Secondary, no heuristic	0.371	0.05	0.421

Continued on next page

Table 5 – continued from previous page

Environment	Malfunction type	Rescheduling type	T_{ground}	T_{solve}	T_{res}
	Consequent + collisions	Base	0.607	3.14	3.747
		Secondary + heuristic	0.467	0.40	0.867
		Secondary + time priority	0.512	0.62	1.132
		Secondary, no heuristic	0.472	2.25	2.722
env_6_6					
	Non-consequent	Base	7.597	131.10	138.697
		Secondary + heuristic	10.564	0.23	10.794
		Secondary + time priority	10.706	168.56	179.266
		Secondary, no heuristic	11.150	245.22	256.370
	Consequent 1	Base	7.927	119.60	127.527
		Secondary + heuristic	9.211	0.73	9.941
		Secondary + time priority	9.466	78.21	87.676
		Secondary, no heuristic	10.102	195.93	206.032
	Consequent 2	Base	8.254	89.68	97.934
		Secondary + heuristic	9.505	0.74	10.245
		Secondary + time priority	8.516	58.83	67.346
		Secondary, no heuristic	10.437	80.04	90.477
	Consequent + collisions	Base	7.346	92.87	100.216
		Secondary + heuristic	5.342	20.51	25.852
		Secondary + time priority	5.776	65.85	71.626
		Secondary, no heuristic	5.322	51.70	57.022
env_10_8					
	Non-consequent	Base	3.874	20.83	24.704
		Secondary + heuristic	4.408	0.10	4.508
		Secondary + time priority	3.997	35.23	39.227
		Secondary, no heuristic	4.116	106.08	110.196
	Consequent 1	Base	3.472	4.48	7.952
		Secondary + heuristic	1.181	0.05	1.231
		Secondary + time priority	0.475	1.26	1.735
		Secondary, no heuristic	1.356	16.62	17.976
	Consequent 2	Base	3.582	21.64	25.222
		Secondary + heuristic	1.194	0.15	1.344
		Secondary + time priority	1.450	3.96	5.410
		Secondary, no heuristic	1.210	29.80	31.010
env_10_10					
	Non-consequent	Base	1.441	43.14	44.581
		Secondary + heuristic	1.273	0.03	1.303
		Secondary + time priority	1.357	8.46	9.817
		Secondary, no heuristic	1.312	82.30	83.612

Continued on next page

Table 5 – continued from previous page

Environment	Malfunction type	Rescheduling type	T_{ground}	T_{solve}	T_{res}
	Consequent 1	Base	1.513	95.72	97.233
		Secondary + heuristic	1.550	0.06	1.610
		Secondary + time priority	1.564	64.74	66.304
		Secondary, no heuristic	1.662	94.84	96.502
	Consequent 2	Base	1.472	0.40	1.872
		Secondary + heuristic	0.479	0.03	0.509
		Secondary + time priority	0.470	0.27	0.740
		Secondary, no heuristic	0.478	0.41	0.888
	Consequent + collisions	Base	1.380	70.34	71.720
		Secondary + heuristic	2.077	0.51	2.587
		Secondary + time priority	1.552	22.53	24.082
		Secondary, no heuristic	1.594	69.77	71.364

Table 5: Running times for different malfunction and rescheduling configurations. Here, T_{ground}, T_{solve}, and T_{res} denote the grounding time, solving time, and total rescheduling time, respectively. All times are reported in seconds.

B Decision process for selecting a rescheduling encoding

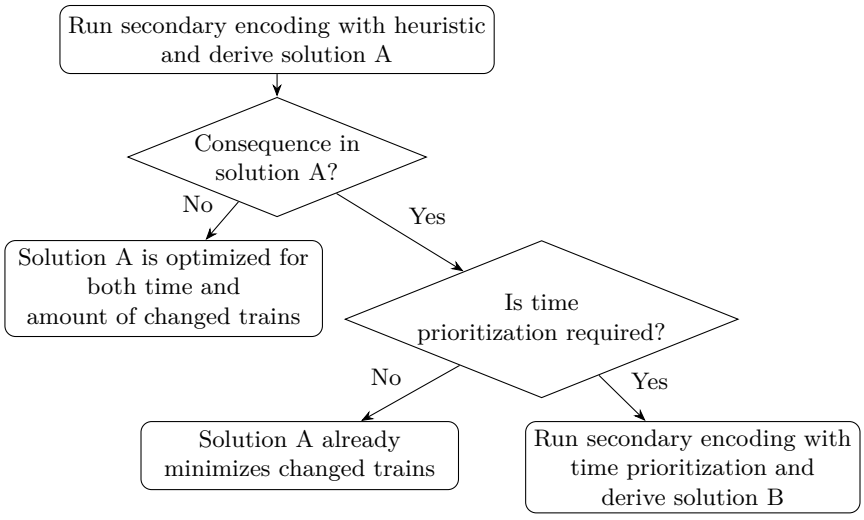


Fig. 5: Decision process for selecting a rescheduling encoding.